The beginning of these tests is similar to the td.when tests. You first create a test-double object using td.constructor, then inject it into the real object before calling the method under test.

At the end of the first test, you call td.verify, which does your mock-object verification. The argument to td.verify is almost completely parallel to that of td.when—the argument to td.verify is the exact function call you want to have been made on the test-double object. All of the argument matchers I talked about for td.when can also be included in td.verify.

The second test differs only in that the final line uses expect().toVerify rather than td.verify. The two lines are functionally equivalent. The expect().toVerify version is denied by the testdouble-jasmine package specifically to convince Jasmine that there's an expectation being tested. If you only use the first version, Jasmine won't realize that you're asserting something and will not count it when calculating the number of assertions. To be clear, the td.verify will work; it just won't create accurate Jasmine metadata.

That test passes. Here's what Task looks like now:

js_jasmine/03/app/javascript/packs/task.js

```
import {TaskUpdater} from "../../../app/javascript/packs/task_updater.js"

export class Task {
  constructor(name, size, id) {
    this.name = name
    this.size = size
    this.project = null
    this.id = id
    this.updater = new TaskUpdater(this)
  }

  index() {
    return this.project.tasks.indexOf(this)
  }

  isFirst() {
    if (this.project) {
      return this.project.firstTask() === this
    }
    return false
  }

  isLast() {
    if (this.project) {
      return this.project.lastTask() === this
    }
    return false
  }
```